University of East Anglia Norwich

School of Computing Sciences

ACADEMIC YEAR	2026-2027
MODULE	CMP-5058A ARTIFICIAL INTELLIGENCE
ASSESSMENT NAME	AI Coursework1
WEIGHTING	60%
SET BY	Dr Farhana Ferdousi Liza
CHECKED BY	Dr Daniel Paredes-Soto
RELEASE DATE	12 October 2026
SUBMISSION DATE	7 December 2026 at 15:00
SUBMISSION LOCATION	Blackboard
FEEDBACK RETURN DATE	11 January 2027
CO-TAUGHT MODULE / WEIGHTING	CMP-6058A — 60%
CO-TAUGHT MODULE / WEIGHTING	CMP-7058A — 60%
CO-TAUGHT MODULE / WEIGHTING	CMP-6058A Final Year Entry — 55%

OVERVIEW & LEARNING OUTCOMES

Context & Scenario:
This group assessment challenges students to build an interactive gaming system where hand gestures replace traditional controls, blending combinatorial game theory with real-time computer vision. By training classifiers on sign language landmarks, students create a hands-free interface for the strategic Hinger game, mirroring real-world applications in accessibility technology and human-computer interaction. The project ultimately demonstrates how AI can bridge physical gestures and digital decision-making in an engaging, practical context.

Learning Outcomes Assessed:

- Apply basic and advanced search methods covered in the module to solve problems in the Hinger game.
- Develop Python programming and problem-solving skills for Artificial Intelligence applications.
- Explain data pre-processing and preparation steps, with emphasis on data classification.
- Demonstrate hands-on experience in collecting data for research and deploying AI tools.
- Design, train, improve, and evaluate supervised learning classifiers.
- Apply programming skills for data analysis, modelling, and effective use of AI libraries.
- Save and test trained AI models in isolated environments for real-time predictions.
- Communicate technical work effectively through presentations, justifying design choices, and answering questions clearly.

EMPLOYABILITY SKILLS

Technical Skills

Git/Version Control, Programming Fundamentals, AI Workflows, Architecture, Testing + QA

Transferable Skills

Communication, Teamwork, Time Management, Critical Thinking, Problem Solving, Ethical Awareness, Adaptability + Resilience

TASK SPECIFICATION

Group Mechanics (Target Size: 4):
All members of a group must belong to the same cohort. For instance, a group formed for CMP-7058A must consist exclusively of members from the CMP-7058A cohort. Please refer to the announcement for further details.

Task Spec / Core Objectives:

Part 1 Overview

The objective of this part of the coursework is to design and implement solutions for a combinatorial game problem known as Hinger, with a focus on comparing the efficiency of different techniques learned in the module. You will collaborate with fellow students (your coursework partners) as a group, subject to approval from the assessment setter. All coding components for this coursework should be part of a single Python project, structured across multiple .py files.
Some tasks in this exercise require some research, particularly around efficient algorithms and better strategies for playing the game.
Each Python file created as part of this assessment should be appropriately commented and must include a header comment that contains your group number, your student ID and your assessment partners' student IDs. You may adapt the project header template header.py in the briefs folder on BlackBoard for use in each module file.

** Python files without a header comment with your group number and student ID will not be marked. **

Description

To support your progress through the coursework, several subtasks are marked with an asterisk (*). These may be deferred to a later stage, as they tend to be more complex and have less impact on subsequent tasks. This structure allows you to prioritise foundational tasks first, while reserving more complex components for when you are better prepared to tackle them.

Hinger Game

The Hinger game is played on a square board consisting of 5 rows and 5 columns. The top-left cell is designated as position (0, 0). Each cell on the board is either empty or active (meaning it contains one counter). Two cells (r,c) and (r',c') are defined as adjacent if $\max\{|r-r'|,|c-c'|\}=1$. This means they are directly next to each other: horizontally, vertically, or diagonally.

An active region is a maximal connected set of active cells, denoted by region Ω, with the following three properties: (i) each cell in Ω is active, (ii) any two cells in Ω are connected by a path of adjacent active cells within Ω, and (iii) all active cells adjacent to any cell in Ω are also included in Ω. For example, State s1 in Figure 1.1 contains two regions: the first region consists solely of the cell at position (0, 4) while the second region consists of all other active cells.

A hinger is a special type of active cell: it has only one counter and when a move is made on a hinger (i.e., the last counter is removed from this cell), the cell becomes empty and the number of active regions on the board increases by one or more. An example of a hinger is highlighted in bold in State s2 of Figure 1.1.

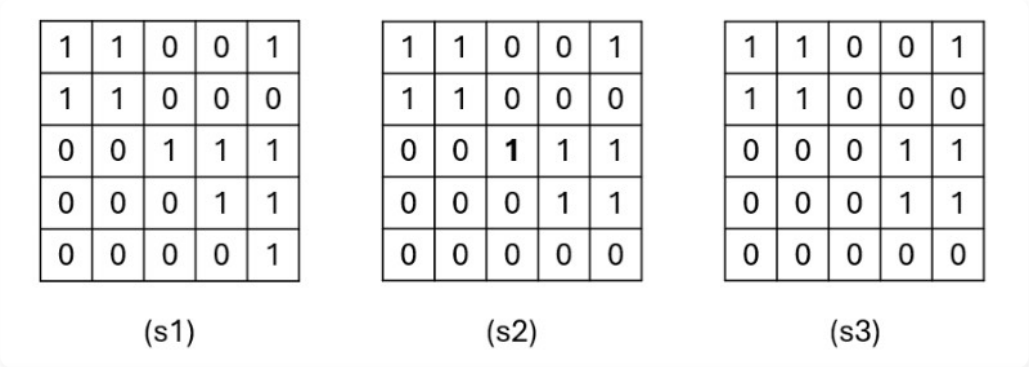


Figure 1.1: Example of three states of the Hinger game. Left (State s1) contains eleven active cells forming two distinct active regions. Middle (State s2) contains a hinger cell (2,2) highlighted in bold. Right (State s3) displays three active regions. In a hypothetical gameplay sequence, player Alice performs a move ‘Y’ (see Figure 1.2) at position (4,4), transitioning the board from State s1 to State s2. Subsequently, player Bob performs a move ‘M’ (see Figure 1.2) at position (2,2), transforming State s2 into State s3. . Alice's move has a bonus of 0 and a cost of 2 because, at the time the move is made, the cell at position (4,4) is adjacent to two active cells. In contrast, Bob's move has a cost of 3, but a bonus of 100 points because the cell at position (2,2) is a hinger when the move is performed.

In Hinger, a single legitimate move consists of removing one counter from an active cell. Each board position is associated with a unique letter from ‘A’ to ‘Y’ (see Figure 1.2), representing the corresponding move. The letter ‘Z’ is reserved as a special action that allows a player to declare the end of the game. The game is played by two players who take turns removing one counter from the board until a player declare the end of the game or all counters are removed. The cost of a move is the number of active cells adjacent to (i,j). The bonus awarded for a move on cell (i,j) is 100 points if the cell (i,j) is a hinger at the time the move is made; otherwise, the bonus is 0 points. At the start of the game, each player is assigned an initial score of 100 points. For every subsequent move made by that player, the move cost is deducted from their score, and any applicable bonus is added. Consequently, a player's final score is calculated as: Final Score = 100 + (total bonuses)−(total costs). The player who plays ‘Z’ loses the game immediately. Otherwise, once all counters have been removed from the board, the game ends and the player with the higher score wins the game. If both players have the same score, the game is declared a draw.

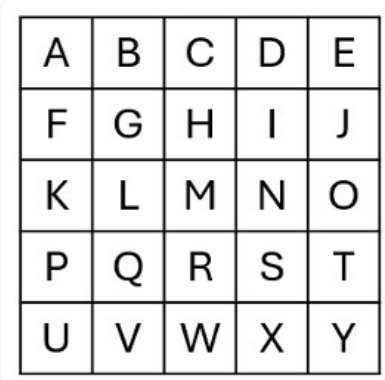


Figure 1.2: Move-encoding scheme for the Hinger game board. Each cell is associated with a unique uppercase letter. For example, a move performed at position (0,0) is encoded by the letter ‘A’. A sequence of moves is represented by a string of letters, ordered chronologically from left to right. For example, the sequence of the two moves illustrated in Figure 1.1 is encoded as "YM", indicating that move 'Y' was made first, followed by move 'M'. The letter 'Z' is reserved as a special action for declaring the end of the game and is therefore not associated with any board position.

Tasks

1. Create a Python module file named a1_state.py within your project folder. In the module file, define a State class that encapsulates the basic structure and behavior of a game state in the game Hinger. The class should include one instance variable named grid representing the number of counters in each cell of the board. Your State class should also have:

   1.1) A proper initialiser, which receives a 2D list grid as a parameter.

   1.2) A sensible **str** method, which returns a readable string representation of the state (e.g., a matrix similar to those in Figure 1.1, without borders).

   1.3) A method named move() that receives a letter representing a move, and perform the corresponding action, provided that the move is legitimate.

   1.4) A generator method named successors() that yields all possible states reachable in one move (i.e., removing one counter from any active cell).

   1.5) A method numRegions() that calculates and returns the number of active regions on the board.

   1.6) A method numHingers() that returns the number of hinger cells in the current state.

   1.7) Any additional utility methods that support gameplay logic or improve code clarity.

   1.8) Within this file, define a test function named tester() to validate your implementation. This function should execute automatically when the script a1_state.py is run directly, but not when imported as a module. Inside this function, implement the following tasks: a. Create a State instance named sa representing State s1 from Figure 1.1, and print its string representation. b. Implement additional testing cases to verify your implementation.

2. Create a module file named a2_agent.py within your project folder. This module will define an Agent class that models a player capable of making strategic decisions in the Hinger game. The agent should be designed to support multiple modes of play and demonstrate intelligent behaviour based on the current game state. The class should include two instance variables: name is a string representing the agent's name (e.g., your group name, such as A9, as the default name) and modes is a list of game-playing strategies available to your agent. Note that Your modes list should contain (i) an element named 'minimax', for which your agent plays using the minimax strategy, and (ii) an element named 'alphabeta', for which the agent plays using the alpha-beta pruning strategy. Your Agent class should also include the following:

   2.1) A proper initialiser, which receives one parameter name, an optional string representing the agent's name, with your group name as the default value. The initialiser should set up the agent accordingly.

   2.2) A sensible **str** method.

   2.3) A method next_move(state, mode) that receives a State object representing the current game board and a playing strategy, and returns a legitimate move for your agent (e.g. a letter according to the scheme in Figure 1.2). If no move is possible (e.g. there are no active cells), return None. Use the best strategy you have implemented as the default value for the mode parameter.

   2.4) Additional variables and supporting methods as needed to facilitate decision-making and strategy implementation for playing a Hinger game.

   2.5) Within this file, create a test function named tester(), which should run automatically when the file a3_agent.py is executed directly (but not when imported). In this function add appropriate code to demonstrate and validate the agent's behaviour across scenarios.

   2.6) [*] Implement other advanced strategies for your agent. For example, you may consider implementing a version of Monte Carlo Tree Search (MCTS).

3. Create a new module file named a3_game.py within your project folder. This module will implement the core gameplay loop for the Hinger game, allowing two agents (or one agent and a human player) to take turns making moves on a shared game state. Within this file, define a function with the signature play(state, agentA, agentB) to simulate a complete game session between two players. It receives a State object representing the initial game board. Furthermore, agentA and agentB are instances of the Agent class (automated player) defined in Task 3, or a human player. For instance, agentA=None indicates that agentA is a human player. The function should perform all of the following tasks:

   3.1) Alternate turns between agentA and agentB.

   3.2) It should detect when the game is over: Declare a win and return the score and name of the winner if one player wins the game, or declare a draw and returns None otherwise.

   3.3) It should detect and handle illegal moves: If an agent or a human player attempts an invalid move (e.g. on an empty cell or non-existence cell), terminate the game and declare the opponent as the winner (e.g. return the name of the winner).

   Your module file should also have:

   a. A proper tester() function to validate your implementation.

   b. [*] Conduct your own research and implement additional features to make the game more engaging. Possible extensions could include: a timing-limited game where each turn has a time limit (e.g., 60 seconds), tracking total time used by each player, counting and displaying the number of moves made during the game, saving the move history to a file for analysis or replay, or implementing a graphical user interface (GUI) for visual gameplay.

Part 2 Overview

This part of the assignment includes collecting data and training supervised learning models to recognise American sign language (ASL) letters. You will deploy trained models to analyse images or a web camera streaming to recognise ASL letters performed by the user allowing them to play the Hinger game in Part 1 interactively. In this part of the assignment, you will train supervised learning algorithms to recognise static ASL letters from ‘A’ to ‘Z’ (as shown in Figure 2.1a) and convert them into one of the possible valid moves to play the Hinger game (refer to Figure 1.2 in Part 1).

Description

Hand pose recognition has many applications. One example is sign language recognition, which involves hand movements, body posture and facial expressions for communication. A simplified version of this task, the recognition of static hand poses, can serve as a basis for developing human-computer interfaces.

Figure 2.1 shows the alphabet in American Sign Language (ASL) and the British Sign Language (BSL). Unlike the BSL, the ASL is simpler to handle by automated recognition systems, because the letters are formed using one hand, which reduces self-occlusions and requires less motion.

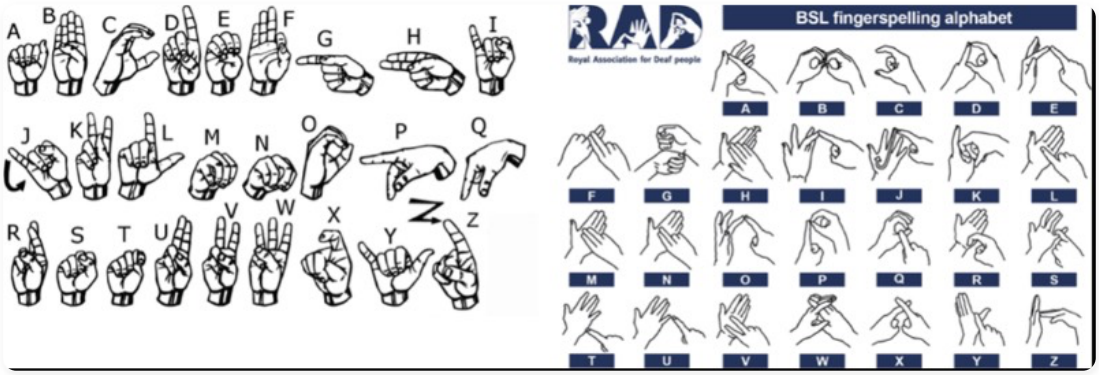


Figure 2.1. Left (a): American Sign Language (ASL), right (b): British Sign Language (BSL) alphabet.

You will not recognise hand poses directly from pictures. Instead, a static hand pose will be represented as a vector of spatial coordinates of landmarks on the hand, such as fingertips and joints. Figure 2.2 shows a typical set of 21 landmarks. Since each landmark has three coordinates, a hand pose is described by a 63-dimensional feature vector.

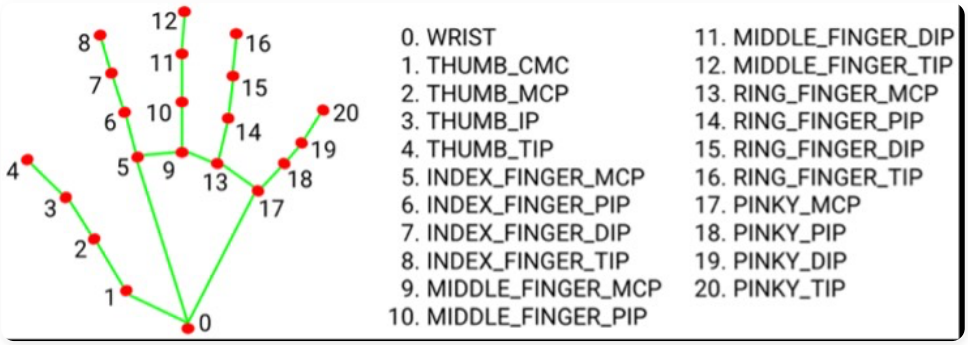


Figure 2.2. Set of landmarks representing the hand geometry for Part 2 and Part 3 of the coursework.

The next sections explain how you will convert an image of a hand pose into such feature vector.

You will train and deploy supervised learning algorithms to recognise static hand poses performed in front of a camera. Then, you will map the recognised pose into one of the valid moves in the Hinger game to make the game interactive.

Tasks

4. Collecting and annotating data: In group, you must collect your own dataset to train supervised learning algorithms. Using any other dataset not approved by the coursework setter is not allowed. Each member of the group should collect 10 images for each of the ASL letters shown in Figure 2.1a, each image will show the static pose for that letter. Follow the steps below:

   4.1) Use a room or space with a plain and light-coloured wall (preferably white) as the background.

   4.2) Use adequate and uniform lighting (ideally natural daylight) in the room.

   4.3) Familiarise yourself with the ASL letters using your right hand.

   4.4) Take 10 pictures of each of the ASL letters, slightly varying the orientation of your hand and the distance from the camera. Feel free to use a webcam or a mobile phone.

   4.5) Ensure that only your right hand appears in the picture. Avoid including objects and shadows on your hand. You may need to move your hand away the wall to prevent shadows around your hand.

   For consistency, all images must be the same size. To achieve this, process the collected images on your computer as follows:

   i) Crop the region covering your hand and save it as a new image. Ensure that the cropped region is a square aspect ratio and do your best to centre the hand in the squared cropped region. You can use the Photos app in Windows for cropping images.

   ii) Resize the cropped images to 256 by 256 pixels. Use a tool of your choice for resizing images.

   iii) Save the 256 x 256 pixel images with the following file naming convention: {studentID}_{ASL-sign}_{example-number}.jpg. For example, the file named 100312345_A_1.jpg corresponds to the first picture taken by the student with ID 100312345 for the letter ‘A’.

   iv) Each student will take 10 pictures for each ASL letter.

   v) Organise the images into folders, with one folder for each ASL letter.

PCs in the lab SCI 0.59 have a built-in web camera, you may find them useful to complete Task 4.

5. From images to feature vectors: Data annotation, or labelling, is a time-consuming task. In this Part 2 of the coursework, for example, annotating the images could be done by using an image editor to manually estimate the position of the 21 landmarks shown in Figure 2.2 and build feature vectors. This process would need to be repeated for every image in the dataset, 260 images per group member that yield to over 5,000 landmarks to pinpoint. How long would it take? Manual annotation is not only slow, but it's also error prone. It might be difficult to accurately pinpointing landmarks due to occlusions, as well as inter– and intra–observer errors. To minimise these problems, you will use the off-the shelf AI tool Google MediaPipe to estimate the (x, y, z) coordinates of the 21 landmarks shown in Figure 2.2.

   For a given image, MediaPipe returns 63 floating-point values that you will arrange into a vector to represent the hand pose in the image. If MediaPipe fails to detect a hand in an image, set the 63 values to zero, and treat the instance as noise when processing the data in Task 6.

   Read the MediaPipe Hand Landmarker documentation to understand the landmark order returned by MediaPipe, and the (x, y, z) coordinate values (x and y are normalised to the width and height, and z is the estimated depth). Store the data in the format of your choice, but a typical choice is a tabular form with one row per hand pose instance, 63 columns for the x, y, z coordinates and an additional column for the class label ('A' – 'Z'). You will be expected to understand and explain your choice of storing the data during the inspection.

6. Cleaning, preprocessing data and feature engineering: You must implement data analysis techniques you consider appropriate to effectively describe the data. For example, describing class distribution and features range of values. You must implement data cleaning and pre-processing steps you consider appropriate, including, but not limited to, identifying and removing noise, outliers and duplicates; and, data encoding.

   Note that hand poses can appear at different positions in the image frame and at different sizes, depending on the distance of the hand with respect to the camera. You may want to transform the data so that hand pose recognition does not depend on where the hand is or how large it appears.

   You must also apply feature engineering techniques, for example, analysing feature importance, dimensional reduction, or creating additional features. The output of the feature engineering step will produce the data used in Task 7.

   Consider creating Python module files with descriptive file names to complete the steps in task 6. Don't forget to document your code.

   You will be expected to understand and explain how you cleaned and prepared the data before using it to train supervised learning algorithms. Use your findings and visualisations to support and justify your choices.

7. Training supervised learning algorithms: Evaluate and compare the performance of at least three supervised classifiers. You must include k-NN and Decision Tree, plus other classifier(s) of your choice. The kNN algorithm must be implemented from scratch using only Python standard built-in libraries (see https://docs.python.org/3.12/library/index.html for a complete list). If you are not sure which libraries are permitted for coding kNN, you should contact the assessment setter for clarification.

   Create a python file named knn.py in your project folder, where you will implement the kNN algorithm. Design your kNN so that it accepts the training and test data in a format that allow you to pass the same data to all classifiers. Similarly, design the kNN so that it reports the same performance metrics as the other classifiers.

   First, split the data into training and test sets; justify your split strategy. Then, train and improve the performance of the chosen classifiers as follows:

   a) Choose suitable performance metrics (e.g., accuracy, sensitivity, and ROC AUC). You will be expected to justify your choice.

   b) Fine-tune at least two common hyperparameters using 5-fold cross-validation on the training set. Make sure you understand the impact of parameters' values. Try as many values as you may need until feeling that you have a good classifier.

   c) Use the default hyperparameter values as a baseline. Report the cross-validation performance on both, the default settings and the best configuration.

   d) Retrain the classifier on the entire training set using the best configuration and report its performance.

   e) Save the trained classifier with the best configuration in the format of your choice. You will use them in Task 8.

   f) Evaluate the retrained classifier on the test set and report its performance.

   g) Use visuals (e.g., tables, plots, and confusion matrices) to report key information, such as class distribution, performance with hyperparameter default values and with the different settings tested, and the performance on the entire training set and test set.

   h) Report the optimisation approach you followed and compare the performance of the classifiers and justify which one you consider the best overall.

   For task 7, create a python file named training_and_optimisation.py, where you will implement the methods that you need, including training and improving classifiers, and producing plots.

8. Deploying the trained classifiers in a human-computer interface: In this task, you will deploy the best classifier (from task 7), in a simple human-computer interface to play the Hinger game using hand pose recognition. You will create a new file a3_advGame.py (based on a3_game.py in Part 1) to use trained classifiers for recognising ASL letters to perform one of the ‘A’ to ‘Z’ valid moves in the Hinger game when the human takes a turn.

   As first step, you must ensure that the trained model is loaded and remain in the memory during the gameplay loop. Create the get_move() method, which should process either a stored test image or a live camera stream to return the ‘A’ to ‘Z’ move–associated to the recognised hand pose–and action the human player moves during its turn.

   For the chosen test image or camera video frame, use MediaPipe to extract landmarks and represent the hand pose as a vector of values. Apply the same preprocessing steps you followed for training classifiers, and enter the resulting feature vector into the classifier in the memory to recognise the ASL letter being posed. Each recognised ASL letter should be mapped to a Hinger move–a cell or special action.

   Consider how your a3_advGame.py should allow the player to load a test image from disk or to process a video frame if work with a camera stream. For example, the user could enter the filename of the image to load, or press a special key (e.g., RETURN) to take a snapshot from the camera and process such image. Integrate the get_move() output into the game loop to demonstrate the use of a trained classifier. You will be expected to explain the design and the choices made during the inspection.

9. **(For CMP-6058A and CMP-7058A only)** Summarising Part 2 in an academic poster: Produce an academic-style poster to summarise your work in Part 2. The aim of a poster is to present a visual abstract of your work, and to disseminate your findings.

   Use MS PowerPoint to design an A1 paper size poster in whichever orientation you prefer. You may need to research how to write an effective academic poster, covering aspects such as layout, structure, text size and colour scheme, visuals and clear and concise academic writing. Additional resources can be found here and the resources provided by the UEA Library listed in the appendix available in BB alongside this coursework specification file. Consider the following guidelines:

   - Typic al sections in a poster include: title, author(s) information, introduction / background, methods, results, conclusions and references and acknowledgements
   - Not all sections can be found in practice. You are free to include, remove or merge sections as you feel that they will be the most effective to communicate your work.
   - It is not essential to include references, but you may use them to support elements that would benefit from adding evidence (e.g., description of the ASL letters, and the Google MediaPipe tool).
   - You may follow this simple guide for font size: title: 100-140pts; headings: 70-100pts; content text: 30-50pts; and figure/table captions, and reference list: 20pts.
   - Aim for a maximum 450 words in the body text.
   - Consider your classmates as the audience when writing to make the poster clear and accessible.

   Save the poster in PDF format and name the file using your group name. For example, use the name A109.pdf for the poster created by the members in the group A109. Ensure that the poster PDF file does not exceed 10MB in size.

Architecture & Technical Constraints:
Python Libraries

Part 1

Only the Python standard libraries (see https://docs.python.org/3.12/library/index.html for a complete list) and Matplotlib library (https://matplotlib.org/) are allowed for Tasks 1 to 2. For Task 3, you may also use the Pygame library (https://www.pygame.org/docs/) for visualisation. If you are not sure which libraries are permitted, you should contact the assessment setter or module organiser for clarification. Your code should be runnable with Python 3.12, and it is your responsibility to ensure that your program runs smoothly on the lab PCs.

Part 2

You can use the Python standard libraries (see https://docs.python.org/3.12/library/index.html for a complete list), Matplotlib or seaborn libraries (https://matplotlib.org/) and other libraries used in the lecture notes and lab sheets. Note that there is a restriction on using libraries to code the kNN algorithm in Task 7. If you are not sure which libraries are permitted, you should contact the assessment setter or module organiser for clarification. It is your responsibility to ensure that your solution runs smoothly on the lab PCs.

DELIVERABLES

Source Code/Files: A single `.zip` file containing all code and a `README.md`.
Documentation/Report: Concisely document the architecture and methods.

The group work will be assessed via an in-person oral presentation, which will be delivered in the week 12. Each student working in a group must submit the collected data, the source code along with a ReadMe file with instruction on how to run the source code, and the GitHub repository link (optional), presentation slides and poster, via the submission point on Blackboard, which will be made available closer to the submission deadline.

All your code should be properly documented to explain the purpose of each function and relevant sections. Create a .zip file named as {your-group-name}_code.zip containing your .py files and the file with the extracted feature data. All code that you submit must be your own. Refer to the section on plagiarism, collusion and contract cheating.

Submission Instructions:

Submit via the Blackboard assignment link before the deadline.

RESOURCES & CONTACT

Contact Information:

For Part 1 related questions, please contact Dr Taoyang Wu (Taoyang.Wu@uea.ac.uk)

For Part 2 related questions, please contact Dr Daniel Paredes-Soto (D.Paredes-Soto@uea.ac.uk)

For general questions (e.g., team grouping), please contact Dr Farhana Ferdousi Liza (F.Liza@uea.ac.uk)

EVALUATION & GRADING

Hinger state class (10%)

- Implementation of methods
- Code quality

Agent (15%)

- Implementation of methods
- Game playing strategies

Hinger game (10%)

- Agents: AI, human player
- Validation of Hinger game rules and actions
- Code quality

ASL data collection (5%)

- Image capture, ASL ‘A’ – ‘Z’ letters
- Image processing and data organisation
- Representation of hand poses with MediaPipe landmarks

Data analysis, pre-processing and feature engineering (10%)

- Data analysis
- Data preprocessing
- Feature engineering and final feature vectors

k-NN classifier (5%)

- Design
- Improvement
- Code quality

Classifiers (Decision Tree, and others) (15%)

- Design
- Improvement

Real-time hand pose recognition and HCI integration (5%)

- Real-time hand pose recognition from unseen images
- Mapping hand poses to Hinger actions
- Integration of a HCI into the Hinger game

Structure, organisation, professionalism and question answering of the inspection, and academic poster (25%)

PLEASE READ ALL THE INFORMATION BELOW CAREFULLY

ACADEMIC INTEGRITY

The University takes academic integrity very seriously. You must not commit plagiarism, collusion, or contract cheating in your submitted work. Our Policy on Plagiarism, Collusion, and Contract Cheating explains:

- what is meant by the terms 'plagiarism', 'collusion', and 'contract cheating'
- how to avoid plagiarism, collusion, and contract cheating
- using a proofreader
- what will happen if we suspect that you have breached the policy.

It is essential that you read this policy, and you undertake (or refresh your memory of) our school's training on this. You can find the policy and related guidance here:

https://my.uea.ac.uk/departments/learningand-teaching/students/academic-cycle/regulations-and-discipline/plagiarism-awareness

In this assessment, working with others is **PERMITTED**. Collaboration is permitted only within your formally allocated group and must follow the group mechanics stated in this brief.

AI POLICY AND USE

To ensure fairness and clarity, this module uses a Traffic Light system to outline exactly how you can and cannot use generative AI tools for your assessment.

🔴 RED	🟡 AMBER ✓	🟢 GREEN

🟡 AMBER: RESTRICTED AI USAGE PERMITTED

Generative AI tools may be used for specific, restricted purposes within this assessment.

Permitted Uses: You may use AI tools to assist with coding and debugging, and to aid your learning

Prohibited Uses: You WILL NOT be able to use AI tools in your presentation, so you will need to understand your work in detail in order to answer questions about it.

Declaration Requirement: *You must explicitly document any allowed AI use. Failure to declare permitted use is considered academic misconduct.*